

Documentacion de un Sistema de Gestion para una Tienda Online

Beatriz Garcia Lopez

13 de julio de 2026

1. Introducción

Este proyecto consiste en diseñar y construir la base de datos completa para gestionar las operaciones de una tienda online, asegurando que la información de productos, clientes y ventas esté organizada y protegida.

2. Descripción del problema

Lo que hace este sistema es resolver inconsistencias en el inventario y las ventas el cual permite registrar qué compró cada cliente, saber exactamente cuántos productos quedan disponibles y evitar que se vendan artículos agotados.

3. Análisis y Diseño

En este apartado vamos a estructurar las tablas e implementar un D-ER (Diagrama Entidad-Relacion) utilizando la página "Draw.io"

TABLAS	¿QUE ORGANIZA?
Categorias	Organiza los productos en 5 categorias diferentes que son atómicas y (teléfonos, laptops, accesorios, tablets y relojes inteligentes)
Reseñas	Registra las opiniones de los productos realizadas por los clientes (las reseñas solo podrán ser realizadas por clientes que hayan adquirido el producto), sin importar la categoría del mismo. Además, contamos con una escala de ranking 1-5 estrellas (siendo 1 = muy malo y 5 = excelente).
Pedidos	Especifica los atributos únicos del pedido por ejemplo: fecha de pedido y el estado del mismo (Enviado, Pendiente y Entregado).
Detalles del Pedido	Guarda los datos únicos del pedido. La cantidad de productos (¿cuántos productos se van a enviar?) y el precio unitario del producto.
Clientes	En esta tabla está toda la información personal del cliente: nombre, correo, teléfono y dirección.
Productos	Esta tabla es el núcleo de las tablas, la cual nos permitirá conectarnos a las demás tablas mediante FK (Foreign Keys). De igual manera, tenemos los atributos: nombre, descripción, precio y stock de todos los productos disponibles al momento.

A continuacion tenemos el Diagrama E-R explicando de mejor manera la tabla

3.1. Diagrama Entidad-Relacion

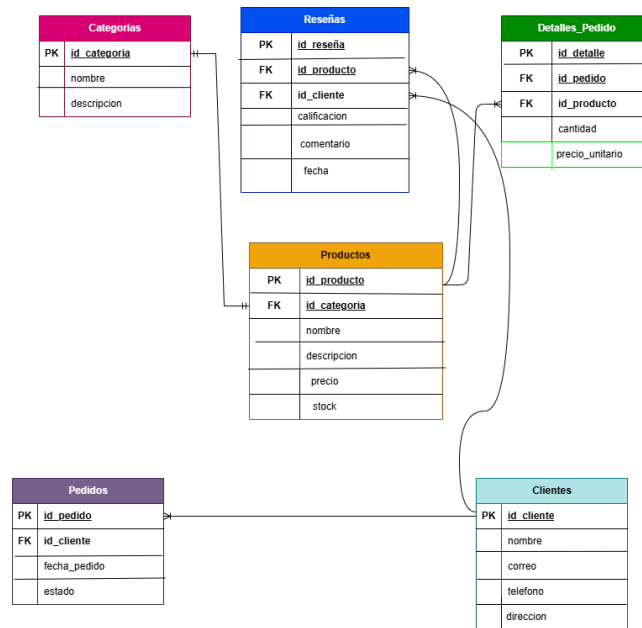


Figura 1: DER de la Base de Datos

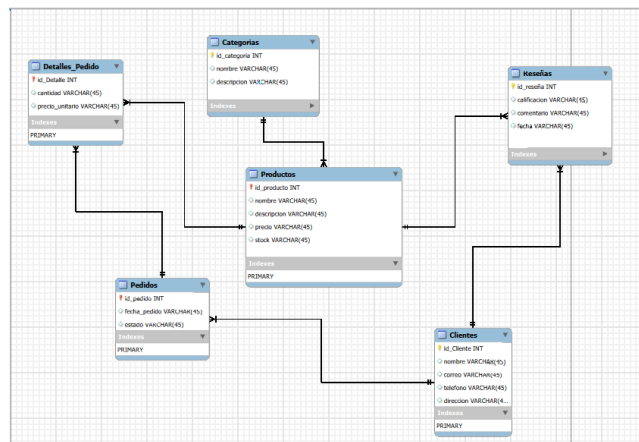


Figura 2: DER de la Base de Datos en Workbench

3.2. Esquema en tercera forma normal (3NF), con justificación de normalización.

Primera Forma Normal (1NF): Todas las tablas tienen datos atómicos. Esto significa que no tenemos listas con datos mezclados en una sola celda; cada campo guarda un solo dato. Por ejemplo, en la tabla **Clientes**, el nombre, correo y teléfono están separados en sus propios campos, en lugar de tener un solo campo saturado de "datos de contacto".

Segunda Forma Normal (2NF): Todos los datos de una tabla dependen por completo de su llave principal(PK). Un buen ejemplo es la tabla **Detalles_Pedido**: la cantidad de productos comprados depende exactamente de esa línea de compra en específico (id_detalle), no solo del pedido ni del producto por sí solos.

Tercera Forma Normal (3NF): Ningún dato depende de otro que no sea la llave principal. Tenemos dos ejemplos:

- En la tabla **Pedidos**, no colocamos el nombre del cliente dentro de la tabla, porque el nombre depende del cliente (por eso tiene su propia tabla), no del pedido en sí. Solo los unimos con la llave foránea id_cliente".

En la tabla **Productos**, no fue necesario escribir el nombre o la descripción de la categoría. Solo debemos utilizar id_categoria. Así, si la categoría cambia de nombre, se actualiza en su propia tabla y no crea errores en la tabla de los productos.

3.3. Identificación de Llaves

A continuación tenemos las llaves primarias(PK), foráneas(FK) y candidatas de cada entidad dentro del modelo relacional del proyecto:

Llaves Primarias (PK)

Las llaves primarias seleccionadas actúan como identificadores únicos para cada registro dentro de su propia tabla:

- **Categorías:** id_categoria
- **Productos:** id_producto
- **Clientes:** id_cliente
- **Pedidos:** id_pedido
- **Detalles_Pedido:** id_detalle
- **Reseñas:** id_reseña

Llaves Foráneas (FK)

Las llaves foráneas tienen una integridad referencial y las relaciones entre las entidades del sistema:

- **Productos:** `id_categoria` (referencia a la tabla `Categorias`)
- **Pedidos:** `id_cliente` (referencia a la tabla `Cientes`)
- **Detalles_Pedido:** `id_pedido` (referencia a la tabla `Pedidos`) y `id_producto` (referencia a la tabla `Productos`)
- **Reseñas:** `id_producto` (referencia a la tabla `Productos`) y `id_cliente` (referencia a la tabla `Cientes`)

Llaves Candidatas

Las llaves candidatas son aquellos atributos que también poseen la cualidad de unicidad y podrían funcionar como identificadores principales, aunque se haya optado por un ID por eficiencia:

- **Cientes:** El atributo `correo` es una llave candidata fuerte, ya que en el sistema de ventas no deberían existir dos clientes registrados con la misma dirección de correo electrónico.
- **Categorias:** El atributo `nombre` funciona como llave candidata, ya que no tendría sentido tener dos categorías llamadas exactamente igual.

4. Implementacion de la Base de Datos

En esta sección se presenta el Script correspondiente a la creación del esquema relacional, la definición de llaves primarias y foráneas, así como las restricciones de integridad y la creación de índices para optimizar las consultas. Finalmente, se incluye el código utilizado para poblar la base de datos con registros de prueba

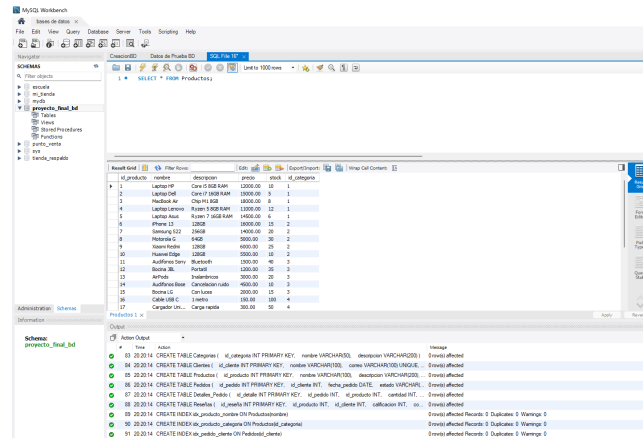
```
1 -- 1. Crear esquema y limpiar versiones viejas
2 CREATE SCHEMA IF NOT EXISTS proyecto_final_bd;
3 USE proyecto_final_bd;
4 DROP TABLE IF EXISTS Resenas, Detalles_Pedido, Pedidos, Productos,
   Cientes, Categorias;
5
6 -- 2. Crear las tablas principales
7 CREATE TABLE Categorias (id_categoria INT PRIMARY KEY, nombre
   VARCHAR(50), descripcion VARCHAR(200));
8 CREATE TABLE Cientes (id_cliente INT PRIMARY KEY, nombre VARCHAR
   (100), correo VARCHAR(100) UNIQUE, telefono VARCHAR(20),
   direccion VARCHAR(200));
9 CREATE TABLE Productos (id_producto INT PRIMARY KEY, nombre VARCHAR
   (100), descripcion VARCHAR(200), precio DECIMAL(10,2), stock
   INT CHECK (stock >= 0), id_categoria INT, FOREIGN KEY (
   id_categoria) REFERENCES Categorias(id_categoria));
```

```

10 CREATE TABLE Pedidos (id_pedido INT PRIMARY KEY, id_cliente INT,
    fecha_pedido DATE, estado VARCHAR(20), FOREIGN KEY (id_cliente)
    REFERENCES Clientes(id_cliente));
11 CREATE TABLE Detalles_Pedido (id_detalle INT PRIMARY KEY, id_pedido
    INT, id_producto INT, cantidad INT, precio_unitario DECIMAL
    (10,2), FOREIGN KEY (id_pedido) REFERENCES Pedidos(id_pedido),
    FOREIGN KEY (id_producto) REFERENCES Productos(id_producto));
12 CREATE TABLE Resenas (id_resena INT PRIMARY KEY, id_producto INT,
    id_cliente INT, calificacion INT, comentario VARCHAR(255),
    fecha DATE, FOREIGN KEY (id_producto) REFERENCES Productos(
    id_producto), FOREIGN KEY (id_cliente) REFERENCES Clientes(
    id_cliente));
13
14 -- 3. Crear indices para agilizar busquedas
15 CREATE INDEX idx_producto_nombre ON Productos(nombre);
16 CREATE INDEX idx_producto_categoria ON Productos(id_categoria);
17 CREATE INDEX idx_pedido_cliente ON Pedidos(id_cliente);
18
19 -- 4. Insertar datos (Muestra representativa)
20 INSERT IGNORE INTO Categorias (id_categoria, nombre, descripcion)
    VALUES (1, 'Laptops', 'Computadoras portatiles');
21 INSERT IGNORE INTO Clientes (id_cliente, nombre, correo, telefono,
    direccion) VALUES (1, 'Juan Perez', 'juan@email.com', '
    5551112222', 'Calle 1');

```

Listing 1: Creación de Base de Datos



```

1      USE proyecto_final_bd;
2
3
4  -- 1. CONSULTAS DE LECTURA (JOIN)
5
6  -- Consulta A: Productos por categoría ordenados por precio
7  SELECT c.nombre AS categoria, p.nombre AS producto, p.precio
8  FROM Productos p
9  JOIN Categorías c ON p.id_categoria = c.id_categoria
10 ORDER BY c.nombre, p.precio;
11
12 -- Consulta B: Clientes con pedidos pendientes
13 SELECT c.nombre, COUNT(p.id_pedido) AS total_pendientes
14 FROM Clientes c
15 JOIN Pedidos p ON c.id_cliente = p.id_cliente
16 WHERE p.estado = 'Pendiente'
17 GROUP BY c.id_cliente;
18
19 -- Consulta C: Top 5 productos mejor calificados
20 SELECT p.nombre, AVG(r.calificacion) AS promedio
21 FROM Productos p
22 JOIN Reseñas r ON p.id_producto = r.id_producto
23 GROUP BY p.id_producto
24 ORDER BY promedio DESC
25 LIMIT 5;
26
27
28
29 -- 2. PROCEDIMIENTOS ALMACENADOS
30
31 DROP PROCEDURE IF EXISTS sp_registrar_pedido;
32 DROP PROCEDURE IF EXISTS sp_registrar_reseña;
33 DROP PROCEDURE IF EXISTS sp_actualizar_stock;
34 DROP PROCEDURE IF EXISTS sp_cambiar_estado_pedido;
35 DROP PROCEDURE IF EXISTS sp_eliminar_reseñas_producto;
36 DROP PROCEDURE IF EXISTS sp_agregar_producto;
37 DROP PROCEDURE IF EXISTS sp_actualizar_cliente;
38 DROP PROCEDURE IF EXISTS sp_reporte_stock_bajo;
39
40 DELIMITER //
41
42 -- 1. Registrar pedido (Valida máximo 5 pendientes)
43 CREATE PROCEDURE sp_registrar_pedido(IN p_cliente INT)
44 BEGIN
45     DECLARE v_pendientes INT;
46     DECLARE v_id INT;
47     SELECT COUNT(*) INTO v_pendientes FROM Pedidos WHERE id_cliente
48         = p_cliente AND estado = 'Pendiente';
49
50     IF v_pendientes < 5 THEN
51         SELECT IFNULL(MAX(id_pedido), 0) + 1 INTO v_id FROM Pedidos
52         ;
53         INSERT INTO Pedidos (id_pedido, id_cliente, fecha_pedido,
54             estado) VALUES (v_id, p_cliente, CURDATE(), 'Pendiente');
55     ELSE
56         SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Límite de
57             pedidos pendientes alcanzado';
58     END IF;
59 END //

```

```

54     END IF;
55 END //
56
57 -- 2. Registrar reseña (Valida que el cliente haya comprado el
    producto)
58 CREATE PROCEDURE sp_registrar_reseña(IN p_prod INT, IN p_cli INT,
    IN p_cal INT, IN p_com VARCHAR(255))
59 BEGIN
60     DECLARE v_compra INT;
61     DECLARE v_id INT;
62     SELECT COUNT(*) INTO v_compra FROM Detalles_Pedido d JOIN
    Pedidos p ON d.id_pedido = p.id_pedido
63     WHERE p.id_cliente = p_cli AND d.id_producto = p_prod;
64
65     IF v_compra > 0 THEN
66         SELECT IFNULL(MAX(id_reseña), 0) + 1 INTO v_id FROM Reseñas
        ;
67         INSERT INTO Reseñas (id_reseña, id_producto, id_cliente,
    calificacion, comentario, fecha) VALUES (v_id, p_prod, p_cli,
    p_cal, p_com, CURDATE());
68     ELSE
69         SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'El cliente no
    ha comprado este producto';
70     END IF;
71 END //
72
73 -- 3. Actualizar stock
74 CREATE PROCEDURE sp_actualizar_stock(IN p_prod INT, IN p_cantidad
    INT)
75 BEGIN
76     UPDATE Productos SET stock = stock - p_cantidad WHERE
    id_producto = p_prod;
77 END //
78
79 -- 4. Cambiar estado de pedido
80 CREATE PROCEDURE sp_cambiar_estado_pedido(IN p_pedido INT, IN
    p_nuevo_estado VARCHAR(20))
81 BEGIN
82     UPDATE Pedidos SET estado = p_nuevo_estado WHERE id_pedido =
    p_pedido;
83 END //
84
85 -- 5. Eliminar reseñas de un producto
86 CREATE PROCEDURE sp_eliminar_reseñas_producto(IN p_prod INT)
87 BEGIN
88     DELETE FROM Reseñas WHERE id_producto = p_prod;
89 END //
90
91 -- 6. Agregar producto evitando duplicados
92 CREATE PROCEDURE sp_agregar_producto(IN p_cat INT, IN p_nom VARCHAR
    (100), IN p_desc VARCHAR(255), IN p_precio DECIMAL(10,2), IN
    p_stock INT)
93 BEGIN
94     DECLARE v_existe INT;
95     DECLARE v_id INT;
96     SELECT COUNT(*) INTO v_existe FROM Productos WHERE nombre =
    p_nom AND id_categoria = p_cat;

```



```

97
98     IF v_existe = 0 THEN
99         SELECT IFNULL(MAX(id_producto), 0) + 1 INTO v_id FROM
100         Productos;
101         INSERT INTO Productos (id_producto, id_categoria, nombre,
102         descripcion, precio, stock) VALUES (v_id, p_cat, p_nom, p_desc,
103         p_precio, p_stock);
104     ELSE
105         SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'El producto ya
106         existe';
107     END IF;
108 END //
109
110 -- 7. Actualizar cliente
111 CREATE PROCEDURE sp_actualizar_cliente(IN p_id INT, IN p_tel
112     VARCHAR(15), IN p_dir VARCHAR(255))
113 BEGIN
114     UPDATE Clientes SET telefono = p_tel, direccion = p_dir WHERE
115     id_cliente = p_id;
116 END //
117
118 -- 8. Reporte stock bajo
119 CREATE PROCEDURE sp_reporte_stock_bajo()
120 BEGIN
121     SELECT nombre, stock FROM Productos WHERE stock < 5;
122 END //
123
124 DELIMITER ;

```

Listing 2: Script Consultas de Lectura

5. Consultas y Procedimientos Almacenados

```

1  -- Consultar clientes con pedidos pendientes
2  SELECT c.nombre, COUNT(p.id_pedido) AS total_pendientes
3  FROM Clientes c JOIN Pedidos p ON c.id_cliente = p.id_cliente
4  WHERE p.estado = 'Pendiente' GROUP BY c.id_cliente;
5
6  -- Procedimiento: Registrar pedido limitando a 5 pendientes
7  DROP PROCEDURE IF EXISTS sp_registrar_pedido;
8  DELIMITER //
9  CREATE PROCEDURE sp_registrar_pedido(IN p_cliente INT)
10 BEGIN
11     DECLARE v_pendientes INT;
12     DECLARE v_id INT;
13     SELECT COUNT(*) INTO v_pendientes FROM Pedidos WHERE id_cliente
14     = p_cliente AND estado = 'Pendiente';
15
16     IF v_pendientes < 5 THEN
17         SELECT IFNULL(MAX(id_pedido), 0) + 1 INTO v_id FROM Pedidos
18         ;
19         INSERT INTO Pedidos (id_pedido, id_cliente, fecha_pedido,
20         estado) VALUES (v_id, p_cliente, CURDATE(), 'Pendiente');
21     ELSE

```

```

19 SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Limite
    alcanzado';
20 END IF;
21 END //
22 DELIMITER ;

```

Listing 3: Lógica de la Tienda Online

```

1  -- Consulta 1: Productos por Categoría
2  • SELECT c.nombre AS categoria, p.nombre AS producto, p.precio, p.stock
3  FROM Productos p
4  JOIN Categorías c ON p.id_categoria = c.id_categoria
5  WHERE p.stock > 0
6  ORDER BY c.nombre ASC, p.precio ASC;

```

categoria	producto	precio	stock
Accesorios	Mica Cristal	100.00	120
Accesorios	Cable USB C	150.00	100
Accesorios	Funda iPhone	200.00	60
Accesorios	Mouse inalámbrico	250.00	50
Accesorios	Cargador Universal	300.00	50
Accesorios	Teclado USB	300.00	40
Accesorios	Base Laptop	400.00	20
Accesorios	Powerbank	500.00	45
Accesorios	Webcam	600.00	30
Accesorios	Microfono	800.00	15
Audio	Bocina JBL	1200.00	35
Audio	Audifonos Sony	1500.00	40
Audio	Bocina LG	2000.00	15
Audio	AirPods	3000.00	20
Audio	Audifonos Bose	4500.00	10
Laptops	Laptop Lenovo	11000...	12
Laptops	Laptop HP	12000...	10
Laptops	Laptop Asus	14500...	6
Laptops	Laptop Dell	15000...	5
Laptops	MacBook Air	18000...	8
Monitores	Monitor Acer	2500.00	25
Monitores	Monitor Samsung	3000.00	20
Monitores	Monitor Dell	3200.00	10
Monitores	Monitor LG	4000.00	15
Monitores	Monitor Asus	4500.00	8
Smartphones	Motorola G	5000.00	30
Smartphones	Huawei Edge	5500.00	10
Smartphones	Xiaomi Redmi	6000.00	25
Smartphones	Samsung S22	14000...	20
Smartphones	iPhone 13	16000...	15

Figura 3: Resultado de la Consulta 1: Productos por categoría.

```

1  -- Consulta 2: Clientes con pedidos pendientes
2  • SELECT c.nombre AS cliente, COUNT(p.id_pedido) AS total_pedidos_pendientes
3    FROM Clientes c
4   JOIN Pedidos p ON c.id_cliente = p.id_cliente
5  WHERE p.estado = 'Pendiente'
6  GROUP BY c.id_cliente, c.nombre;

```

cliente	total_pedidos_pendientes
Ana Martinez	1
Diego Morales	1
Ricardo Vargas	1
Carlos Lopez	1

Figura 4: Resultado de la Consulta 2: Clientes con pedidos pendientes.

```

1  -- Consulta 3: Top 5 productos con mejor calificación
2  • SELECT p.nombre AS producto, AVG(r.calificacion) AS promedio_calificacion
3    FROM Productos p
4   JOIN Reseñas r ON p.id_producto = r.id_producto
5  GROUP BY p.id_producto, p.nombre
6  ORDER BY promedio_calificacion DESC
7  LIMIT 5;

```

producto	promedio_calificacion
Laptop HP	5.0000
Audifonos Sony	5.0000
Laptop Dell	5.0000
MacBook Air	5.0000
iPhone 13	4.0000

Figura 5: Resultado de la Consulta 3: Top 5 productos mejor calificados.

```

1  • USE proyecto_final_bd;
2  -- Procedimiento 1: Registrar un nuevo pedido exitoso, el cual inserta
3  -- el pedido 21 para el cliente 1
4  • CALL sp_registrar_pedido(21, 1, 1, 1, 26);

```

162 | 21.24.58 | CALL sp_registrar_pedido(21, 1, 1, 1, 26) | 1 row(s) affected

Figura 6: Resultado del Procedimiento 1: Registrar un nuevo pedido exitoso

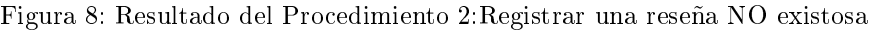
```

1  • USE proyecto_final_bd;
2  -- Procedimiento 2: Registrar una reseña exitosa
3  • CALL sp_registrar_resena(12, 1, 1, 5, 'Me encanto mi nueva laptop');

```

169 | 21.35.37 | CALL sp_registrar_resena(12, 1, 1, 5, 'Me encanto mi nueva laptop') | 1 row(s) affected

Figura 7: Resultado del Procedimiento 2: Registrar una reseña exitosa



```
1 • USE proyecto_final_bd;
2   -- Procedimiento 4: Cambiar el estado de un pedido
3   -- Cambiaremos el pedido 4 de "Pendiente" a "Enviado".
4 • CALL sp_cambiar_estado_pedido(4, 'Enviado');
```

Output		
Action Output		
#	Time	Action
✓ 1	22:09:47	USE proyecto_final_bd
✓ 2	22:09:47	CALL sp_cambiar_estado_pedido(4, 'Enviado')

Figura 10: Resultado del Procedimiento 4: Cambiar el estado de un Producto

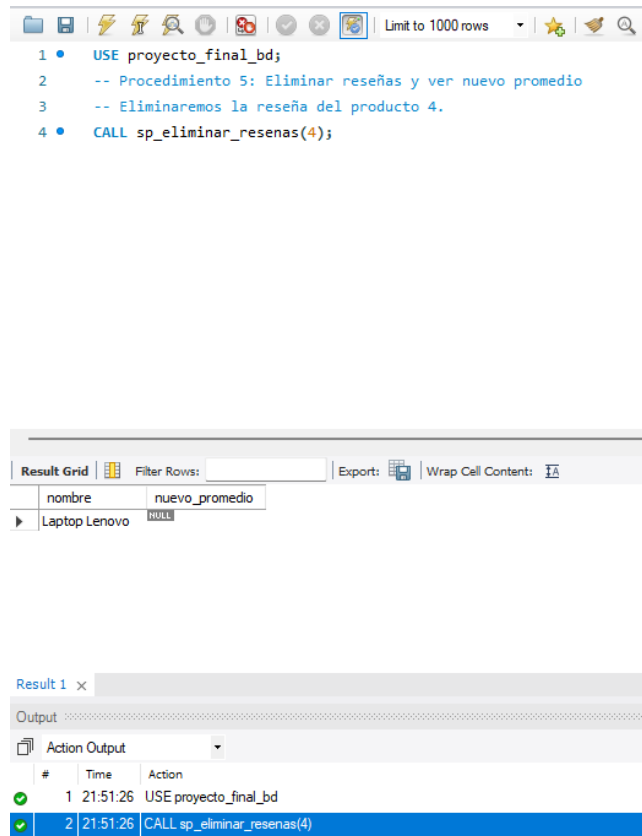


Figura 11: Resultado del Procedimiento 5: Eliminar reseñas y ver nuevo promedio de calificaciones

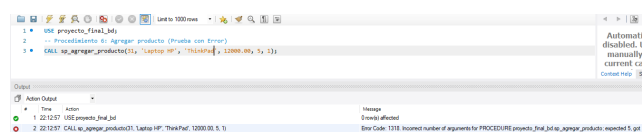


Figura 12: Resultado del Procedimiento 6: Agregar un nuevo producto (Con fines prácticos este procedimiento va a dar ERROR)

1	•	USE proyecto_final_bd;
2		-- Procedimiento 7: Actualizar datos de cliente
3	•	CALL sp_actualizar_cliente(1, 'Avenida Bruno Diaz 8087', '5559998888');

#	Time	Action	Message
1	22:17:58	USE proyecto_final_bd	0 row(s) affected
2	22:17:58	CALL sp_actualizar_cliente(1, 'Avenida Bruno Diaz 8087', '5559998888')	1 row(s) affected

Figura 13: Resultado del Procedimiento 7: Actualizar los datos de un cliente

1	•	USE proyecto_final_bd;
2		-- Procedimiento 8: Reporte de stock bajo
3	•	CALL sp_reporte_stock_bajo();

nombre	stock
Memoria USB 16GB	1

#	Time	Action
1	22:33:35	USE proyecto_final_bd
2	22:33:35	CALL sp_reporte_stock_bajo()

Figura 14: Resultado del Procedimiento 8: Reporte de Stock Bajo

6. Validación y Optimización

Para comprobar que la lógica funciona, ejecutamos el procedimiento de reporte de stock mediante el comando CALL, obteniendo los datos esperados.

```

1  USE proyecto_final_bd;
2
3  -- Prueba de procedimiento 1: Reporte de stock bajo (menor a 5)
4  CALL sp_reporte_stock_bajo();
5
6  -- Prueba de procedimiento 2: Registrar un nuevo pedido exitoso
7  CALL sp_registrar_pedido(1);
8
9  -- Prueba de Optimización: Uso de Índices (Comprobación)

```

```
10 EXPLAIN SELECT * FROM Productos WHERE nombre = 'Laptop HP';
```

Listing 4: Script Validacion y Optimizacion

```
1 • USE proyecto_final_bd;
2 -- Procedimiento 1: Regitara un nuevo pedido exitoso, el cual inserta
3 -- el pedido 21 para el cliente 1
4 • CALL sp_registrar_pedido(21, 1, 1, 1, 26);
```

✓ 162 | 21:24:58 | CALL sp_registrar_pedido(21, 1, 1, 1, 26) | 1 row(s) affected

Figura 15: Ejecución exitosa del procedimiento almacenado (Reporte de stock).

Para validar la optimización, usamos la instrucción `EXPLAIN`. La base de datos utilizó correctamente nuestro índice `idx_producto_nombre`, encontrando la Laptop en una sola lectura (**rows** = 1).

```
9 -- Prueba de Optimización: Uso de índices (Comprobación)
10 • EXPLAIN SELECT * FROM Productos WHERE nombre = 'Laptop HP';
```

id	select_type	table	partitions	type	possible_keys	key	key_len
1	SIMPLE	Productos	idx_producto_nombre	ref	idx_producto_nombre	idx_producto_nombre	403

Result 9 Result 10 ×

Output

Action Output

#	Time	Action
✓ 1	21:52:26	USE proyecto_final_bd
✓ 2	21:52:26	CALL sp_reporte_stock_bajo()
✓ 3	21:52:26	CALL sp_registrar_pedido(1)
✓ 4	21:52:26	EXPLAIN SELECT * FROM Productos WHERE nombre = 'Laptop HP'

Figura 16: Uso de índices en búsqueda rápida.

7. Errores y Correcciones

Problema	Causa	Solución
Error 1054: Unknown column	Nombres de llaves foráneas automáticos y confusos.	Estandarizar la creación de tablas con nombres limpios (ej. <code>id_cliente</code>).
Warning 1366	Insertar datos sin especificar el orden de las columnas.	Declarar las columnas explícitamente en cada inserción.
Error 1452: Foreign key fails	Guardar detalles de un pedido antes que el producto.	Respetar la jerarquía insertando primero los datos padre (categorías, productos).
Error 1318: Incorrect arguments	Actualizar un procedimiento sin borrar el viejo de la memoria.	Usar <code>DROP PROCEDURE IF EXISTS</code> al inicio.

Cuadro 1: Resolución de problemas técnicos.

8. Conclusión

Desarrollar este sistema me permitió entender que una base de datos no es solo guardar información, sino conectarla con lógica real. Aprendí a estructurar las tablas usando normalización, a proteger la información con restricciones y a automatizar procesos con procedimientos almacenados, logrando un proyecto totalmente funcional con las especificaciones establecidas.

9. Material de Apoyo Consultado

- Monroy, G. (2026). *Procedimientos Almacenados, Vistas y Triggers en MySQL*. Material de clase.
- Parzibyte - programador. (s. f.). <https://parzibyte.me/>
- Ivan. (2011, 25 diciembre). Crear BD en MySQL Workbench: Guía paso a paso para principiantes MySQL YA. Mysql YA. <https://mysqlya.com.ar/tutoriales/crear-bd-en-mysql-workbench/>
- Andrés, J. (s. f.). DISEÑO y CREACIÓN DE BASE DE DATOS CON MYSQL WORKBENCH (BD INVENTARIO). <https://base-de-datos-online.blogspot.com/2016/01/disen-y-creacion-de-base-de-datos-con.html>

- Parzibyte. (2021, 20 enero). MySQL - Inner join con count usando subconsulta. Parzibyte. <https://parzibyte.me/blog/posts/mysql-inner-join-count-subconsulta/>
- Tipo de letra en LaTeX | Manualdelatex.com. (s. f.). <https://manualdelatex.com/tutoriales/tipo-de-letra>

10. Usuario de GitHub

bettylp905